أكاديمية كاوست
KAUST ACADEMY

Cybersecurity & Blockchain

KGSP Enrichment [Week 3 & 4 – 2026]

جامعة الملك عبدالله للعلوم والتقنية
King Abdullah University of Science and Technology

Mudassar Aslam
(FHEA, PhD Cybersecurity)

Associate Professor of Cybersecurity & Program Director,
School of Engineering, Computing and Mathematics,
Oxford Brookes University,
Oxford, United Kingdom.
maslam@brookes.ac.uk

www.mudassir.info

Day 08 – Web Application Hacking

Hacking and Pentesting

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

Transitioning to Web Application Hacking

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

Quick Recap

أكاديمية كاوست
KAUST ACADEMY

- The Journey So Far: We have mastered the fundamental phases of hacking machines and networks:
  - Footprinting & Reconnaissance
  - Scanning & Enumeration
  - Exploitation
  - Privilege Escalation
- The New Frontier: While the structured approach remains similar, we now focus on the Web Environment, which requires specialized tools and a shift in mindset from services to logic.

Network vs. Web App Pen-Testing

أكاديمية كاوست
KAUST ACADEMY

Traditional Network Pen-Testing	Web Application Security
Focuses on Operating Systems, network protocols (TCP/IP), and open services (SSH, FTP, SMB)	Revolves around HTTP/HTTPS traffic, how the front-end interacts with the back-end, and flaws in "Business Logic" (how the app is designed to work)
We often look for unpatched services	We often look for custom code vulnerabilities and misconfigurations.

Unique Web Attack Surface

أكاديمية كاوست
KAUST ACADEMY

- Web applications introduce specific entry points that do not exist in traditional network services
  - Input Fields: Every form, search bar, and login page is a potential gateway for injection.
  - Session Management: How the server remembers you - using cookies and tokens - can be hijacked.
  - Databases: Web apps rely on SQL/NoSQL queries; if improperly handled, these can be manipulated.
  - APIs: REST and GraphQL interfaces often expose internal data if not secured correctly.

Phase 1 – Reconnaissance & Fingerprinting

- The goal is to identify the "Technology Stack" without necessarily being detected.
- Identify Technologies: Discover the CMS (WordPress, Joomla) or frameworks (React, Django) being used.
  - Tools: Wappalyzer (browser extension), BuiltWith, or the Proxy in Burp Suite.
- Endpoint Mapping: Finding all the "hidden" pages and entry points of the application.
  - Tools: OWASP ZAP, DirBuster, or FFuf for automated discovery.

Phase 2 – Scanning & Enumeration

أكاديمية كاوست
KAUST ACADEMY

- Moving from passive observation to active discovery.
- Directory Brute-Forcing: Using wordlists to find hidden directories like /admin or /config.
  - Tools: Gobuster or DirBuster.
- Traffic Analysis: Intercepting the "conversation" between your browser and the server to see what data is being sent.
  - Tools: Burp Suite (the industry standard) or OWASP ZAP.

Phase 3 – Exploitation

أكاديمية كاوست
KAUST ACADEMY

- SQL Injection (SQLi): Manipulating database queries to bypass logins or "dump" user data.
- Cross-Site Scripting (XSS): Injecting malicious scripts (JavaScript) into pages viewed by other users.
- Cross-Site Request Forgery (CSRF): Tricking a logged-in user into performing an action (like changing a password) without their knowledge.
- Authentication Bypass: Finding "backdoors" or flaws in the login logic to gain access without valid credentials.

Phase 4 – Post-Exploitation

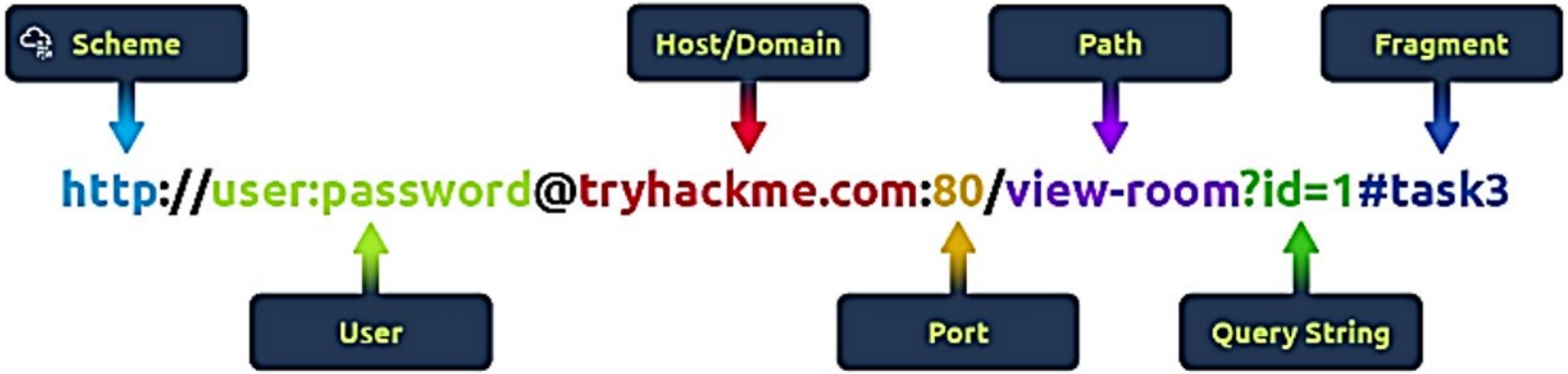
أكاديمية كاوست
KAUST ACADEMY

- What happens after you get "in"?
- Session Hijacking: Stealing a valid user's cookie to "become" them without needing their password.
- Privilege Escalation: Finding ways to go from a "Standard User" to an "Administrator".
- Data Exfiltration: Safely (and ethically) demonstrating that sensitive data can be removed from the system

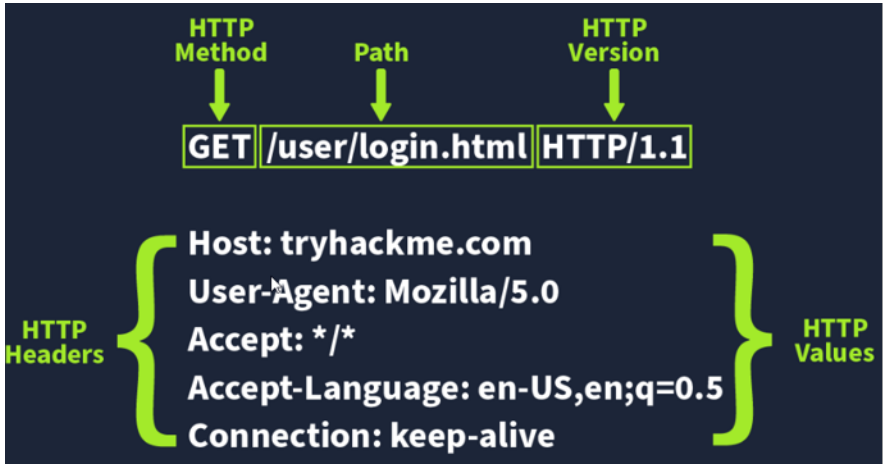
Introduction to Web Applications (revisit)

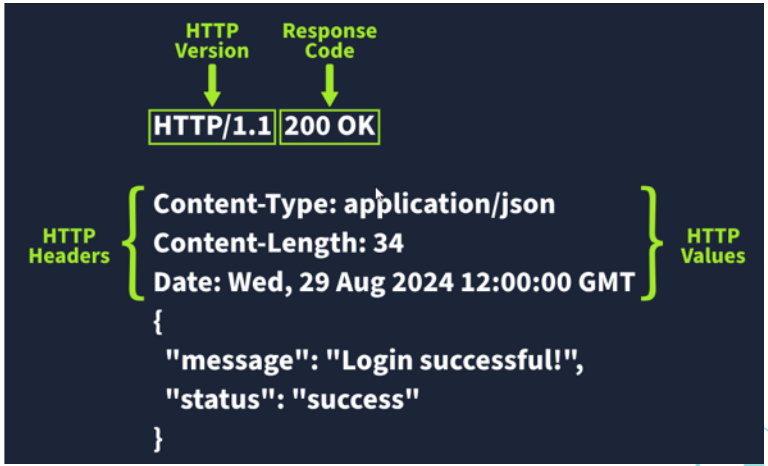
جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

What is a URL? (Uniform Resource Locator)



HTTP Request -> Response





HTTP Methods

أكاديمية كاوست
KAUST ACADEMY

- GET Request: This is used for getting information from a web server.
- POST Request: This is used for submitting data to the web server and potentially creating new records
- PUT Request: This is used for submitting data to a web server to update information
- DELETE Request: This is used for deleting information/records from a web server.

HTTP Status Codes (visit http.cat)

أكاديمية كاوست
KAUST ACADEMY

Code Ranges	Purpose	Common Codes
100-199 Information Response	Request has been accepted; Client should continue sending the rest of their request	No longer very common.
200-299 Success	This range of status codes is used to tell the client their request was successful.	**200 – OK**: The request was completed successfully. **201 – Created**: A resource has been created (for example a new user)
300-399 Redirection	These are used to redirect the client's request to another resource. This can be either to a different webpage or a different website altogether.	**301 - Moved Permanently:** This redirects the client's browser to a new webpage or tells search engines that the page has moved somewhere else and to look there instead. **302 – Found:** Similar to the above permanent redirect, but as the name suggests, this is only a temporary change and it may change again in the near future.
400-499 Client Errors	Used to inform the client that there was an error with their request.	**400 - Bad Request:** Something was either wrong or missing in their request. For example, a requested web server resource also expected a certain parameter that the client didn't send. **401 - Not Authorised:** You are not currently allowed to view this resource until you have authorised with the web application, most commonly with a username and password. **403 – Forbidden**: You do not have permission to view this resource whether logged in or not. **405 - Method Not Allowed:** The resource does not allow this method request, for example, you send a GET request to the resource /create-account when it was expecting a POST request instead. **404 - Page Not Found:** The page/resource you requested does not exist.
500-599 Server Errors	This is reserved for errors happening on the server-side and usually indicate quite a major problem with the server handling the request.	**500 - Internal Service Error:** The server has encountered some kind of error with your request **503 - Service Unavailable:** This server cannot handle your request as it's either overloaded or down for maintenance

Common Request Headers (1/2)

أكاديمية كاوست
KAUST ACADEMY

- Host: Some web servers host multiple websites so by providing the host headers you can tell it which one you require, otherwise you'll just receive the default website for the server.

- User-Agent: This is your browser software and version number, telling the web server your browser software helps it format the website properly for your browser and also some elements of HTML, JavaScript and CSS are only available in certain browsers.

- Content-Length: When sending data to a web server such as in a form, the content length tells the web server how much data to expect in the web request. This way the server can ensure it isn't missing any data.

Common Request Headers (2/2)

- Accept-Encoding: Tells the web server what types of compression methods the browser supports so the data can be made smaller for transmitting over the internet.

- Cookie: Data sent to the server to help remember your information

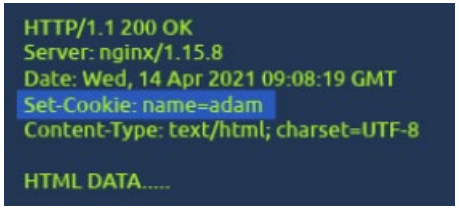
Common Response Headers

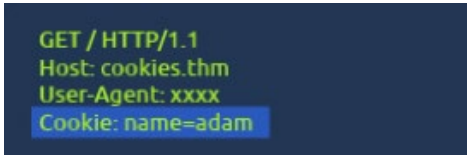
أكاديمية كاوست
KAUST ACADEMY

- Set-Cookie: Information to store which gets sent back to the web server on each request
- Cache-Control: How long to store the content of the response in the browser's cache before it requests it again.
- Content-Type: This tells the client what type of data is being returned, i.e., HTML, CSS, JavaScript, Images, PDF, Video, etc. Using the content-type header the browser then knows how to process the data.
- Content-Encoding: What method has been used to compress the data to make it smaller when sending it over the internet.

Cookies

- Small piece of data stored on the client-side
- Server asks client to save cookies (set-cookies)
- Client sends this cookies in subsequent requests
- Used to remind the web server
  - who you are (authentication token)
  - some personal settings for the website
  - whether you've been to the website before





- Set-Cookie: sessionId=abc123secure; HttpOnly; Secure; SameSite=Strict
  - HttpOnly: Prevents client-side scripts (like JavaScript) from accessing the cookie, which helps mitigate Cross-Site Scripting (XSS) attacks.
  - Secure Only use secure channel to send cookies
  - SameSite=Strict: Provides defense against Cross-Site Request Forgery (CSRF)

Security Headers (https://securityheaders.com/)

- Provide mitigations against attacks like Cross-Site Scripting (XSS), clickjacking, and data injection.

1. Content Security Policy (CSP)
2. Strict-Transport-Security (HSTS)
3. Referrer-Policy
4. X-Frame-Options
5. X-Content-Type-Options
6. Permissions-Policy

1. Content Security Policy

- Protection against XSS (restricts malicious code/scripts to run from a separate website/domain)
- Content-Security-Policy: default-src 'self'; script-src 'self' https://cdn.tryhackme.com; style-src 'self'
  - default-src: specifies the default policy of self, which means only the current website.
  - script-src: specifics the policy for where scripts can be loaded from, which is self along with scripts hosted on https://cdn.tryhackme.com
  - style-src: specifies the policy for where style CSS style sheets can be loaded from the current website (self)
  - img-src: specifies the policy for where images can be loaded from

2. HTTP Strict-Transport-Security (HSTS)

- The HSTS header ensures that web browsers will always connect over HTTPS
- Strict-Transport-Security: max-age=63072000; includeSubDomains; preload
  - max-age=63072000 : how long (in seconds) the browser should remember this rule. 63,072,000 seconds equals exactly 2 years. Once a user visits your site via HTTPS and receives this header, their browser will internally redirect all future "http://" requests to "https://" for the next two years, even if they type "http://" in the address bar. Every time the user visits the site, this timer resets back to 2 years.
  - includeSubDomains: a blanket rule for your entire domain ecosystem. It applies the HSTS policy not just to example.com, but also to api.example.com, blog.example.com, and any other subdomain. Only use this if you are 100% sure that every single one of your subdomains has a valid SSL/TLS certificate. If one old internal tool is still on HTTP, this header will break it for your users.
  - Preload: HSTS only works *after* the first visit (the "Trust On First Use" problem). If a hacker intercepts that very first connection, they can strip the HSTS header. By adding preload, you are signaling to browser vendors (Google, Apple, Mozilla) that you want your domain hardcoded into the browser's source code as "HTTPS-only." Once you are on the "HSTS Preload List," a browser will never even try an HTTP connection, even if the user has never visited your site before.

3. Referrer-Policy

- It tells the browser how much information about the current page should be sent to the next website when a user clicks a link.
- Referrer-Policy: <Directive>

Directive	What it shares	Use Case
no-referrer	**Nothing.** The Referer header is omitted entirely.	Maximum privacy (e.g., a banking dashboard).
same-origin	**Full URL** for your own site, **Nothing** for others.	Good for internal tracking without leaking data to others.
strict-origin	**Domain only** (e.g., https://example.com/), but only via HTTPS.	Tells others *who* sent them traffic, but not *which page*.
strict-origin-when-cross-origin	**Full URL** internally, **Domain only** externally.	**Modern Default.** Best balance for most websites.
unsafe-url	**Full URL** everywhere, even over insecure HTTP.	**Do not use.** High risk of leaking sensitive data.

Webservers

أكاديمية كاوست
KAUST ACADEMY

- A software that listens for incoming connections and then utilises the HTTP protocol to deliver web content to its clients.
- Popular webservers: Apache, Nginx, IIS and NodeJS
- A Web server delivers files from what's called its root directory (defined in the software settings)
- Nginx and Apache: /var/www/html
- IIS: C:\inetpub\wwwroot
- Example:
  - http://www.example.com/picture.jpg is loaded from the local drive at /var/www/html/picture.jpg
- Web servers can host multiple websites with different domain names
- Backend and Scripting languages allow to present dynamic content
  - PHP, Python, Ruby, NodeJS, Perl and many more

THM Rooms

DNS in Detail: https://tryhackme.com/room/dnsindetail

HTTP in Detail: https://tryhackme.com/room/httpindetail

How Websites work: https://tryhackme.com/room/howwebsiteswork

Putting it all together: https://tryhackme.com/room/puttingitalltogether

+

Web Application Basics: https://tryhackme.com/room/webapplicationbasics

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

Walking an Application

Web Reconnaissance & Manual Discovery

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

Web application Reconnaissance

أكاديمية كاوست
KAUST ACADEMY

- Moving beyond automated tools to manually explore a web application's structure and hidden components
- Objectives:
  - Understand the importance of manual "walking" in web pentesting.
  - Identify hidden directories and sensitive files (e.g., .robots.txt, .sitemap.xml).
  - Learn to inspect page source and developer tools for information leakage.
  - Recognize common frameworks and their default configurations

Why Manual "Walking"?

أكاديمية كاوست
KAUST ACADEMY

- The Human Edge: Automated scanners often miss logic flaws or hidden comments left by developers.

- Context Discovery: Understanding how different parts of the application interact (e.g., how a 'Contact' form links to a backend API).

- Stealth: Manual browsing is less likely to trigger Intrusion Detection Systems (IDS) compared to aggressive automated brute-forcing

Initial Footprinting

أكاديمية كاوست
KAUST ACADEMY

- Robots.txt: A file meant for search engines that often lists "Disallowed" directories which are prime targets for hackers.

- Sitemap.xml: Provides a complete map of the website's structure, including pages not linked in the main navigation.

- Favicons: The small icon in the browser tab. Its hash can be used on platforms like Shodan to identify the specific framework (e.g., WordPress, Drupal) being used.

- Headers: Analyzing HTTP response headers (e.g., X-Powered-By, Server) to identify software versions.

Inspecting the Source Code (view source)

- Developer Comments: Often contain "TODO" notes, internal IP addresses, or even hardcoded credentials (e.g., ``).
- Hidden Links: Finding URLs or endpoints that are commented out but still active.
- Javascript Files: Analyzing .js files to find API keys, internal endpoints, or logic that can be manipulated.

Leveraging Browser Developer Tools

أكاديمية كاوست
KAUST ACADEMY

- Inspector: Modify HTML/CSS on the fly to see how the app handles changes.
- Debugger: Set breakpoints in Javascript to see how data is processed before being sent to the server.
- Network Tab: Watch every request and response. Check for sensitive data being sent in plain text (e.g., session cookies, tokens).
- Storage Tab: Inspect Cookies, Local Storage, and Session Storage for potential session hijacking targets.

Identifying Frameworks and CMS

أكاديمية كاوست
KAUST ACADEMY

- Frameworks and CMS have their specific Indicators, such as:
  - WordPress: Look for /wp-admin/ or /wp-content/.
  - Bootstrap: Specific class names in the HTML.
- Once a framework is identified, you can look up known vulnerabilities (CVEs) for that specific version.
- "Wappalyzer" is a browser extension to automate this identification.

Ethical & Legal Reminder

أكاديمية كاوست
KAUST ACADEMY

- Rules of Engagement (RoE): Only "walk" applications you have explicit permission to test.
- White Hat Ethics: Use these skills for defense, not damage.
- Legal Risks: Unauthorized access, even "just looking," can violate the Computer Misuse Act.

Automated Scanning

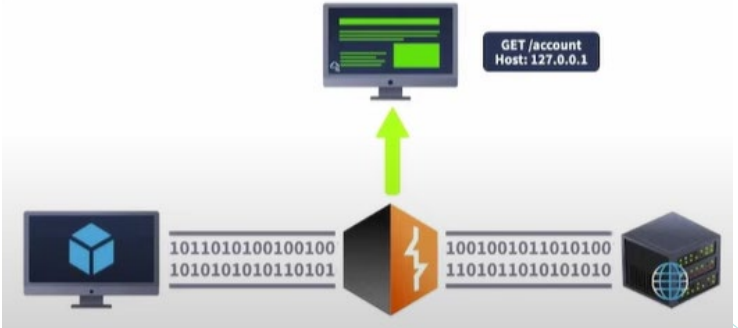
Burp Suite

OWASP ZAP

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

What is Burp Suite?

- The Industry Standard: Burp Suite is the "Swiss Army Knife" for web application penetration testing.
- Core Function: It acts as an Interception Proxy between your browser and the target web server.
- Capabilities:
  - Intercept and modify HTTP/S requests and responses.
  - Automate repetitive attacks (Intruder).
  - Decode and encode data (Decoder).
  - Scan for vulnerabilities (Professional version).



Key Components of Burp Suite

أكاديمية كاوست
KAUST ACADEMY

- Dashboard: Monitor scan progress (Pro) and event logs.
- Target: Define the scope of your engagement to avoid testing unauthorized sites.
- Proxy: The most used tool. It allows you to pause, inspect, and change traffic in real-time.
- Repeater: Manually resend and tweak a single request multiple times to test server behavior.
- Intruder: Used for automated attacks like brute-forcing or fuzzing (limited in Community Edition).

Setting Up the Environment

أكاديمية كاوست
KAUST ACADEMY

- The Proxy Listener: By default, Burp listens on 127.0.0.1:8080.
- Browser Configuration: Manual Proxy settings in Firefox/Chrome.
- FoxyProxy: A browser extension recommended for switching proxies on/off quickly.
- Burp's Built-in Browser: Modern versions of Burp include a pre-configured Chromium browser that requires no setup.

Intercepting Traffic (The Proxy)

أكاديمية كاوست
KAUST ACADEMY

- Intercept is On: The request hangs in Burp. The server has not received it yet.
- The Power of Manipulation
  - Change a User: guest cookie to User: admin
  - Modify a price field in a shopping cart from $100 to $1.
- Intercept is Off: Traffic flows through Burp to the server without stopping, but is still logged in the HTTP History.

The Repeater

أكاديمية كاوست
KAUST ACADEMY

- If you have found an interesting request in your History, "Send to Repeater" (Ctrl+R)
- Iterative Testing: Change one parameter at a time and hit "Send" to see the immediate response.
- Use Case: Testing for SQL Injection by adding a single quote ' to a parameter and observing if the server returns a 500 Error.

The Intruder (automating requests)

أكاديمية كاوست
KAUST ACADEMY

- While Repeater is for manual testing, Intruder is used for high-volume, automated requests.
- The Workflow
  - Define where the "payloads" (input data) should be inserted into the HTTP request (e.g., a password field).
  - Payloads: Supply a list of data, such as a wordlist of common passwords or a sequence of numbers.
  - Attack: Burp sends a request for every item in your list and logs the results.
- Common Attack Types:
  - Sniper: The most common. It takes one wordlist and tests one position at a time.
  - Battering Ram: Puts the same token into multiple positions simultaneously.
  - Pitchfork: Uses multiple wordlists (e.g., List A for usernames, List B for passwords) and pairs them $1:1$.
  - Cluster Bomb: Tests every possible combination of multiple wordlists (exhaustively).

The Intruder – Use cases

أكاديمية كاوست
KAUST ACADEMY

- Brute-forcing: Guessing login credentials using a list like rockyou.txt (see note below)
- Fuzzing: Sending strange characters to find "hidden" server errors or SQLi vulnerabilities.
- Enumeration: Cycling through User IDs (e.g., id=1, id=2) to find valid accounts.

- Note: In the Burp Suite Community Edition, Intruder is "throttled" (speed-limited). This is an intentional restriction to encourage the purchase of the Professional version for enterprise use.

The Decoder

- Web traffic often uses encoding (like Base64 or URL encoding) to transport data safely. To an ethical hacker, this looks like gibberish (e.g., YWRtaW4= instead of admin).
- Decoder allows you to quickly transform data between its raw form and various encoded/hashed formats without leaving Burp Suite
- Work Flow:
  - Intercept request
  - Decoding: Convert encoded strings (Base64, URL, Hex, Octal, HTML) back into "Plaintext.“
  - Encoding: Reconvert the modified plaintext into original format.
  - Forward request
- Use Cases:
  - Session Cookies: If you see a cookie like session=dXNlcl9pZD00Mg==, you can paste it into Decoder to find it decodes to user_id=42. You can then encode a new ID and swap it in the Proxy
  - SQL Injection: Encoding your 'OR 1=1-- payload as a URL string to ensure special characters don't break the HTTP request.
- Smart Decoding: Use the "Decode as... Smart decode" feature to let Burp guess the encoding layers automatically.

The Sequencer (analysing randomness)

أكاديمية كاوست
KAUST ACADEMY

- To determine the unpredictability (entropy) of "random" tokens issued by a web server, such as Session IDs, Password Reset Tokens, or CSRF Tokens.
- If a server generates tokens in a predictable way (e.g., sequentially or based on a timestamp), an attacker can "guess" the session ID of another user and hijack their account.
- Work Flow:
  - You send a request that generates a token (like a login page or a cookie-setting request) to Sequencer.
  - Burp automatically sends the request hundreds or thousands of times to collect a large sample of tokens.
  - Analyze: Burp runs statistical tests (FIPS 140-2) on the bits of the tokens to see if they are truly random.
  - Interpreting Results:
    - High Entropy: The tokens are truly random; the application is secure against token prediction.
    - Low Entropy: The tokens follow a pattern. An attacker could use a script to generate the "next" valid session ID

Burp Suite Tools Summary

أكاديمية كاوست
KAUST ACADEMY

Tool	Purpose	Lab Practice Task
Proxy	Intercept & View	Capture the initial login request.
Repeater	Manual Tweak	Test if changing a UserID field works.
Intruder	Automation	Brute-force the password using `rockyou.txt`.
Decoder	Translation	Base64 decode a hidden cookie value.
Comparer	Analysis	Find the difference between a "Pass" and "Fail" login.
Sequencer	Randomness	Check if the Session ID is guessable.

HTTPS and CA Certificates Troubleshooting

- HTTPS encrypts traffic, preventing Burp from seeing the data
- You must install Burp’s CA Certificate into your browser.
- This allows Burp to perform a "Man-in-the-Middle" (MITM) on your own connection so it can decrypt, show you the data, and then re-encrypt it for the server.

Setting the Target and Scope control

- The Target tab is where Burp organizes everything it discovers about your application into a structured hierarchy.
- The Site Map
  - Hierarchical View: Displays all URLs, folders, and scripts discovered during your "walk" in a tree structure.
  - Visual Cues: Items in Bold have been requested; items in *Grey* have been inferred (linked but not yet visited).
  - Content Discovery: Right-click a folder to "Guess" hidden files (Pro feature) or manually map out the application structure.
- Scope (The "Rules of Engagement")
  - Precision Testing: By adding the Target_IP to your scope, you tell Burp to ignore traffic from other tabs (like YouTube or Google) in your browser.
  - Benefits:
    - Keeps the HTTP History clean.
    - Ensures you don't accidentally attack a third-party site (Legal/Ethical Protection).
    - Enables "Show only in-scope items" filters.
- Issue Definitions
  - A comprehensive list of all vulnerabilities Burp can detect (SQLi, XSS, etc.).
  - Educational Resource: Each entry provides a description of the vulnerability, the impact, and the remediation.

Web Application Vulnerabilities

OWASP Top 10

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

Weaknesses (CWEs) vs Vulnerabilities (CVEs)

- CWE (Common Weakness Enumeration):
  - Think of this as the blueprint for a mistake. It describes a software flaw or error in design, code, or architecture (e.g., "Improper Input Validation").
  - Focus is on Prevention and Education
- CVE (Common Vulnerabilities and Exposures):
  - A specific, documented occurrence of a weakness in a real-world product (e.g., CVE-2021-44228 aka Log4Shell)
  - Focus is on Patching and Remediation
- CWEs lead to CVEs
  - A developer makes a mistake (CWE), and if that mistake is found in a released software (like Microsoft Word or a specific website), it is assigned a CVE ID.

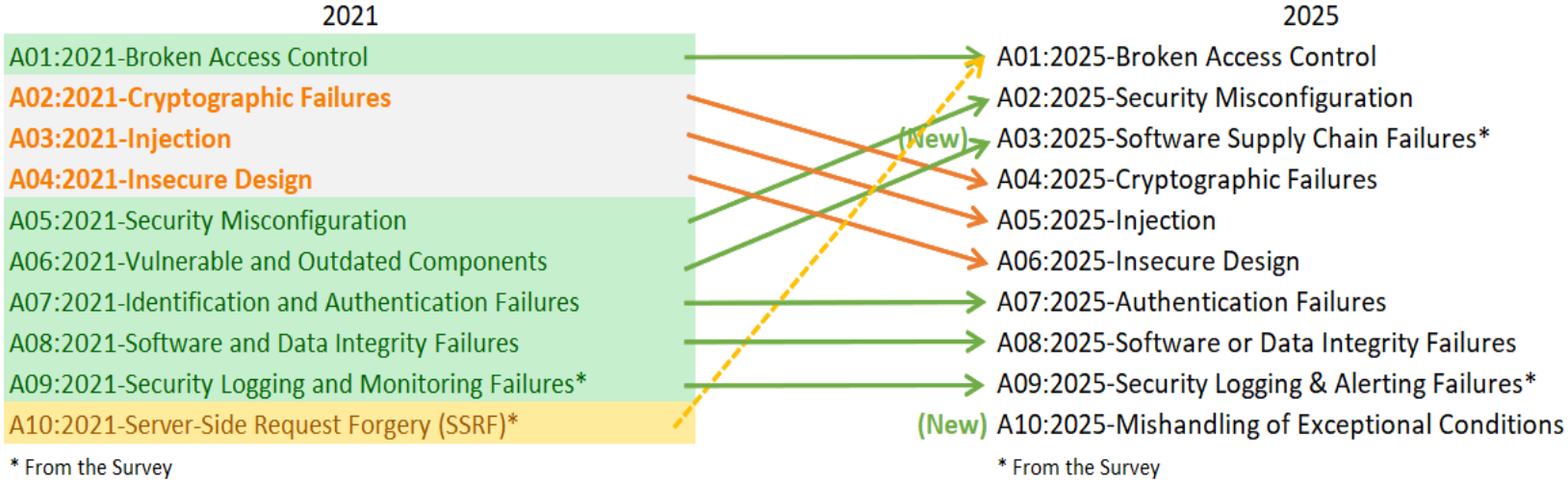
Web Application Weaknesses & OWASP Top 10

- Unlike a private server in a network, web apps are designed to be public-facing and accessible from anywhere.
- Every input field, URL parameter, and cookie is a potential entry point for an attacker
- Security goal is to move focus from “functional code” to “secure code”
- OWASP: The Open Web Application Security Project (OWASP) is a nonprofit foundation that works to improve the security of the web applications.
- OWASP Top 10:
  - It is a regularly updated report outlining the 10 most critical web application security risks.
  - It is globally recognized Industry Standard as the first step toward more secure coding.
  - OWASP doesn't release a new list every year because the landscape takes time to shift significantly. Historically, updates happen every 3-4 years (e.g., 2013, 2017, 2021, and now the 2025 preview).
  - Source: https://owasp.org/Top10/2025/

OWASP Top 10: 2021 vs. 2025

أكاديمية كاوست
KAUST ACADEMY

- Web application security is a moving target. OWASP updates the Top 10 approximately every 3-4 years to reflect modern development trends (Cloud, DevOps, APIs).



A01: Broken Access Control

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

Fundamentals

أكاديمية كاوست
KAUST ACADEMY

- Authentication (PortSwigger Learning Path)
- Session Management
  - Once authenticated, you don't need to re-authenticate for each webpage/resource access
  - A session cookie (token) is used which is sent by each user request to prove that the user is authenticated.
- Access Control
  - Whether user is allowed to carry out the action that are attempting to perform
  - Backend DB checks the access privileges of that user ID (identified by the cookie)
- Vertical Access Control (user vs admin)
- Horizontal Access Control (user1 vs user2)
- Context-Dependent Access Control (initiate transaction->ask for confirmation-< execute)
- Read Further: https://portswigger.net/web-security/access-control

A01:2025 – Broken Access Control

أكاديمية كاوست
KAUST ACADEMY

- Failure to enforce restrictions on what authenticated users are allowed to do.
- The core issue is that the application relies on "Security by Obscurity" or client-side checks rather than server-side verification.
- Impact: Confidentiality, Integrity, Availability
- Broken Access Control is ranked #1 on the OWASP Top 10 because it has the highest "Incidence Rate" in modern web apps. 94% of applications tested by OWASP showed some form of broken access control.

Types of Broken Access Control

- Horizontal Privilege Escalation
  - Original: website.com/account?id=123
  - Broken: website.com/account?id=1000
- Vertical Privilege Escalation
  - Original: website.com/login/home.jsp?admin=false
  - Broken: website.com/login/home.jsp?admin=true
- Context-dependent Access Control vulnerability
  - Original: step 1: confirm -> step 2: execute
  - Broken: step 2: execute -> step 1: change

Exploitation Techniques

أكاديمية كاوست
KAUST ACADEMY

- Change REQUEST by modifying parameters in the URL or HTML page
- Accessing the API with missing access controls on POST, PUT and DELETE methods
- Manipulating metadata, such as replaying or tampering with JSON Web Token (JWTs) or a cookie.
- Force browsing authenticated pages as an unauthenticated user

- More about Broken Access Control:
  - https://owasp.org/Top10/2025/A01_2025-Broken_Access_Control/
  - Broken Access Control | Complete Guide by Rana Khalil (40 min video)

A01:2025 – Access Control (broken) – Labs Tasks

Lab 1: Unprotected admin functionality

Lab 2: Unprotected admin functionality with unpredictable URL

Lab 3: User role controlled by request parameter

Lab 4: User role can be modified in user profile

Lab 5: URL-based access control can be circumvented

Lab 6: Method-based access control can be circumvented

Lab 7: User ID controlled by request parameter

Lab 8: User ID controlled by request parameter, with unpredictable user IDs

Lab 9: User ID controlled by request parameter with data leakage in redirect

Lab 10: User ID controlled by request parameter with password disclosure

A01:2025 IDOR

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

Insecure Direct Object References (IDOR)

أكاديمية كاوست
KAUST ACADEMY

- It is a sub-category of Broken Access Control (A01:2025) where an application uses user-supplied input to access objects directly.
- Core Weakness: The application trusts the user's input (like an ID) to fetch a resource without verifying if the user is actually authorized to see that specific resource.
- Mechanism:
  - The user requests a resource (e.g., GET /user/123).
  - The application takes the "123" and queries the database/filesystem.
  - The Flaw: The server checks if the user is logged in, but fails to check if the user owns record 123.
  - Result: An attacker changes "123" to "124" and views another user's private data.
- IDOR may allow horizontal or vertical privilege escalation

IDOR - Examples

- Example 1 - Direct Database References
  - A website displays account details via a vulnerable URL parameter.
  - https://insecure-website.com/customer_account?customer_number=132355
  - Attacker identifies the customer_number follows a predictable pattern (incremental).
  - Attacker iterates through numbers: 132356, 132357, etc.
  - Impact: Massive data breach of customer records.
- Example 2 - Static File References
  - Web chat transcripts are stored as .txt files on the server.
  - Vulnerable URL: https://insecure-website.com/static/12144.txt
  - The attacker notices the filename is numeric.
  - By changing the filename in the URL, the attacker can download private chat logs of other users.
  - Risk: Often reveals sensitive info like passwords, PII, or internal business logic.

Finding IDOR Vulnerabilities

أكاديمية كاوست
KAUST ACADEMY

- Where to look:
  - Query strings (URLs)
  - POST body parameters
  - Cookie values
  - Headers (e.g., Custom-User-Id: 55)
- Tip:
  - Don't just look for 1, 2, 3.
  - Look for:
    - Hashed IDs (MD5 of a username).
    - UUIDs/GUIDs (Longer strings, but sometimes leak in other API calls).
  - Sometimes an IDOR exists in a "Password Reset" or "Update Profile" function, allowing an attacker to change someone else's password.
- Scanning
  - Use Intruder/Fuzzer to automate the testing of ID ranges.
  - Manual Inspection: Analyze how the app references objects (IDs, UIDs, filenames).

Prevention Strategies

أكاديمية كاوست
KAUST ACADEMY

- Avoid Direct Mapping: Use indirect references (e.g., a temporary session-based map).
- Access Control Checks: Crucial. Every time a resource is accessed, the server must verify: "Does the current session user have permission for this specific object ID?“
- Use Non-Predictable IDs: Implement long, random strings (UUIDs) so attackers cannot guess or "brute-force" IDs.

A01:2025 - IDOR - References

أكاديمية كاوست
KAUST ACADEMY

- A01 Mapped IDOR CWEs
  - CWE-639: Authorization Bypass Through User-Controlled Key
- More Learning resources
  - https://portswigger.net/web-security/access-control/idor
  - https://owasp.org/www-community/attacks/insecure_direct_object_reference

A01:2025 – IDOR – Lab Task

Lab 1: Insecure direct object references

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

A01:2025 - Path Traversal (1)

أكاديمية كاوست
KAUST ACADEMY

- Path traversal is also known as directory traversal. These vulnerabilities enable an attacker to read arbitrary files on the server that is running an application.
- Application code
  - find "hidden" routes, poorly implemented security checks (/var/www/app/main.py or /var/www/html/index.php)
- Application data
  - database connection strings, API keys, and internal server configurations (/var/www/html/config/php.ini or web.config)
- Credentials for back-end systems.
  - AWS Access Key and Secret Key (/home/webapp/.aws/credentials)
  - Private SSH key (/home/webapp/.ssh/id_rsa)

A01:2025 - Path Traversal (2)

أكاديمية كاوست
KAUST ACADEMY

- Sensitive operating system files.
  - Linux User List (/etc/passwd)
  - Linux Password Hashes (/etc/shadow)
  - Windows Registry/Boot (C:\Windows\win.ini or C:\boot.ini)
- In some cases, an attacker might be able to write to arbitrary files on the server, allowing them to modify application data or behavior, and ultimately take full control of the server.
  - Upload a Web Shell (upload shell.php or shell.py to /var/www/html/uploads/)
  - Overwriting Authorized Keys (write own Public SSH Key into the server's filesystem at: /home/webapp/.ssh/authorized_keys)

A01:2025 - Path Traversal - References

- A01 Mapped Path Traversal CWEs
  - CWE-22 Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')
  - CWE-23 Relative Path Traversal
  - CWE-36 Absolute Path Traversal

- More Learning resources
  - https://owasp.org/www-community/attacks/Path_Traversal

A01:2025 - Path Traversal – Labs Tasks

Lab 1: File path traversal, simple case

Lab 2: File path traversal, traversal sequences blocked with absolute path bypass

Lab 3: File path traversal, traversal sequences stripped non-recursively

Lab 4: File path traversal, traversal sequences stripped with superfluous URL-decode

Lab 5: File path traversal, validation of start of path

Lab 6: File path traversal, validation of file extension with null byte bypass

Done with Cybersecurity Training?

If you think you know it better now, what will you do in this situation?

Save Me

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology